2025 International Conference on Communication, Computing, Networking, and Control in Cyber-Physical Systems (CCNCPS) | 979-8-3315-9713-9/25/$31.00 ©2025 IEEE | DOI: 10.1109/CCNCPS66785.2025.11135898

CAPFUL - Coherent Dynamic Controller Placement Method using deep reinforcement learning for SDN

Chaithanya Kurra[1], Venkata Prudhvi Dadi[2], Avinash Awasthi[2], Vijay Janyani[1], and Ramesh Babu Battula[2]

[1]Dept.of E.C.E., Malaviya National Institute of Technology Jaipur, India
[2]Dept.of C.S.E., Malaviya National Institute of Technology Jaipur,India

***Abstract*—Software-defined networking (SDN) has revolutionized intelligent network monitoring and management by splitting the control plane from the data plane, and it has gained momentum to resolve the inherent challenges of traditional networks. The placement of controller nodes to assure load balancing and efficient network & cost management is one of the primary challenges of SDN - the proposed novel Deep Reinforcement Learning (DRL) to solve the SDN load-balanced controller placement challenge. The DRL agent learns from the network's current state, including traffic load and indulged cost, and places or reassigns controller switch mapping accordingly by leveraging the ability to discover optimal controller placement strategies by exploring the solution space and adapting to the dynamic network conditions. We demonstrate the effectiveness and efficacy of our DRL-based Actor-Critic technique in achieving load balancing and improving overall network performance through extensive simulations and experiments. The experiment's outperforming results provided the performance of the proposed DRL in solving complex optimization problems in SDN environments and offered practical guidance for controller placement in real-world networks.**

***Index Terms*—SDN, Controller Placement Problem, optimization, Deep Reinforcement Learning.**

I. Introduction

Software-defined networking (SDN) is emerging as a new network paradigm shift. With its intelligent hierarchical architecture, SDN offers a novel approach to managing and controlling networks. SDN decouples the control plane responsible for network intelligence and policy decisions from the data plane, which handles data forwarding. This separation introduces a centralized, software-based SDN controller that programs underlying network devices through open protocols like OpenFlow, NETCONF, etc. The separation of the control plane (Control Layer) from the data plane (Infrastructure Layer) is a fundamental principle of SDN architecture [1] [2]. The detailed system design architecture can be envisioned in Fig 1. The centralized control performs dynamic network management, and programmability enables network administrators to optimize network behavior and adapt to changing requirements. The SDN architecture uses a single controller to manage the network with programmability. However, single-point failure, lacks of control on the entire network, and latency are the significant challenges. In this regard, the multiple controller concept is transpire to SDN. The multiple control node placement allows greater flexibility and agility in network management [3] [4]. A physically distributed but

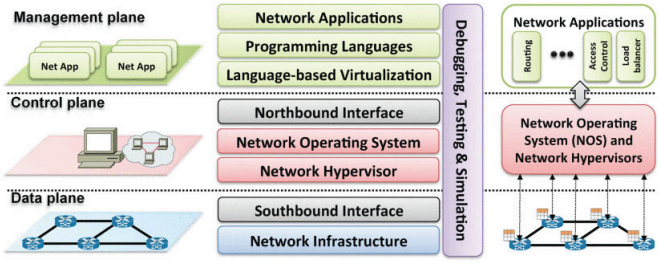


Fig. 1. Hierarchical Architecture of Software-Defined Network

logically connected control plane refers to a network architecture where the SDN controllers are not all located in the same physical location, but they work together as a single, unified entity to manage the entire In Software-Defined Networking, controller placement can be done using static or dynamic strategies. Static deployment means the number and location of controllers are fixed in advance and usually don't change unless manually updated [5] [6] [7]. It's simple and works well for networks that don't change much. On the other hand, dynamic deployment adjusts controller placement based on current network conditions, traffic, and load, which helps with better performance and scalability [8] [9]. It can automatically adapt as the network grows or changes. Sometimes, a hybrid approach combines static and dynamic methods to get the advantages of each, depending on what the network needs [10] [11].

The load-balanced controller placement is a predominant problem in Software-Defined Networking (SDN), which poses a significant challenge in effectively distributing controller nodes to achieve load balancing and optimize network management [7] [2]. Traditional approaches based on heuristics or mathematical models often fail to adapt to dynamic network conditions and varying workload demands, leading to suboptimal controller placement. There is a need for an intelligent and adaptive solution that can address the dynamic nature of network traffic patterns and workload fluctuations. Therefore, the problem statement of this study is to develop a Deep Reinforcement Learning (DRL) based approach that can effectively and autonomously determine the optimal placement of controllers in SDN environments, leading to load balancing, effective cost, and enhanced overall network performance.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:06:27 UTC from IEEE Xplore. Restrictions apply.

The proposed Deep Reinforcement Learning (DRL)is an effective solution for optimization problems in software-defined networks (SDNs) with the multi controller. DRL learns optimal policies by interacting with the environment and receiving rewards for actions that meet network goals, such as reduced latency and improved throughput. Its ability to handle uncertainties and non-linear dynamics makes it superior to traditional methods. Specifically, actor-critic algorithms are employed for dynamic controller placement where the actor selects placement actions and the critic evaluates them. These components are neural network-based and trained using policy gradient methods. Despite challenges in managing large state-action spaces, this approach enhanced scalability and performance.

II. Literature Review

The Software Defined Networking (SDN) providing a promising solutions to the limitations of traditional network architectures by decoupling the control and data planes. The control plane, managed by SDN controllers, governs the behaviour of the data plane through the installation of flow rules in the switches' flow tables. These tables typically comprise three key fields - RULE, ACTION, and STAT, and are dynamically updated via packet-in messages when unmatched flows are detected [3]. A significant challenge in SDN deployment arises with the Controller Placement Problem (CPP), particularly in large-scale networks with multiple controllers. CPP is an NP-hard problem with strong correlations to classical combinatorial optimization problems like the Facility Location Problem and Vertex Cover Problem. The central objective is determining controllers' optimal number and placement to achieve performance goals such as reduced latency and balanced load distribution.

While early CPP solutions focus primarily on static placement strategies optimized for latency [3], [4] or load balancing [8], they often overlook the dynamic nature of real-world traffic, leading to imbalanced controller loads. The Dynamic Controller Placement Problem (DCPP) has been introduced, enabling adaptive switch-to-controller assignments based on network conditions [12]. Advanced strategies such as the controller migration framework [13] and flow redirection methods offer reactive solutions once degradation occurs. However, these techniques are often limited by their post-facto nature. Hence, there is a growing need for proactive and predictive models capable of adjusting controller placements and switch mappings pre-emptively. In this context, Deep Reinforcement Learning (DRL) presents a viable direction by facilitating policy learning through continuous interaction with dynamic network environments. The DRL-based models have shown potential to hadle multi-objective optimizations, including latency minimization and load balancing while adapting effectively to real-time network traffic fluctuations.

III. Proposed Methodology - CAPFUL

In the proposed model, we assume multiple controllers distributed across the network. The data transfers in the network are also assumed to be dynamic. This framework examines two objectives simultaneously. The first is the workload distribution among the controllers, and the second is the associated cost. Each vertex of the graph of n nodes formulated as an SDN Network is either a switch or controller. All these n vertices are interlinked with each other. The graph $G = (V, E)$ denotes the network's topology. V is the set of switches and controllers, and E is the set of tangible connections between these switches or controllers. $d(u, v)$ is the shortest path between nodes u and v $(u, v \in V)$, denoting the distance between nodes u and v. Without sacrificing generality, we presume the switches comprise the set.

$$S = \{w_1, w_2, \ldots, w_n\}$$

have already been deployed in the network at fixed locations. Let k $(k \in \mathbb{N})$ represent the number of controllers in the network, and let

$$C = \{c_1, c_2, \ldots, c_k\}$$

represent their set. The most flow requests that the controller c_j can process concurrently are denoted by B_{max}, which is the controller's capacity.

Assume that 10% of all flows originating from a switch at a given time will be packet-in messages. The number of packet-in flow messages for a switch i is represented by λ_i. Assume C_i represents the set of switches controlled by the controller c_i $(i \in \mathbb{N}, i \leq k)$.

$$C_1 + C_2 + \ldots + C_k = W, \tag{1}$$

$$\text{and} \quad C_i \cap C_j = \varnothing \quad \text{for } i, j \in \mathbb{N},\ i \neq j,\ i, j \leq k \tag{2}$$

since each switch can only be controlled by a single controller. C_j, and

$$C_j = \{w_i^j\} \quad (i, j \in \mathbb{N},\ i \leq n,\ j \leq k)$$

demonstrates that the switch w_i is controlled by the controller C_j. The controller's load will vary based on the flow fluctuation in the network environment. The flow requested by each switch will install the controller that corresponds to it.

We assume that the current burden of each controller is the sum of all flows it processes. We assume a general 5% loss in packet-in message transmission. With this, the load on a controller i can be defined as the load of all the switches assigned to it, that is:

$$L_i = 0.95 \times \sum_{j \in C_i} \lambda_j \tag{3}$$

To assure network performance, the load on each controller must not exceed B_{max}:

$$L_i \leq B_{\text{max}} \tag{4}$$

The average load L_{avg} on all the controllers is:

$$L_{\text{avg}} = \frac{1}{k} \sum_{i=1}^{k} L_i \tag{5}$$

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:06:27 UTC from IEEE Xplore. Restrictions apply.

The mean square deviation ΔL of all the controllers will be the load distribution factor of the network system:

$$\Delta L = \sqrt{\frac{1}{k}\sum_{i=1}^{k}(L_{\text{avg}} - L_i)^2} \tag{6}$$

A good load balance will be seen if ΔL is lower and vice versa. The cost of the controller will be the amount of effort needed to communicate between the interfaces, especially from switches to controllers (data to control plane) and controllers to controllers (within the controller plane).

The weight of the switch wt_i for the i-th switch will be the amount of participation that this switch i has among all the switches:

$$wt_i = n \times \frac{\lambda_i}{\sum_{j=1}^{n}\lambda_j} \tag{7}$$

If $d(u, v)$ is the shortest distance between u and v, then the cost of the controller H_i for controller i will be the cost of all weighted distances of switches under the controller i to controller i:

$$H_i = \sum_{j \in C_i} d(w_j, c_i) \times wt_j \tag{8}$$

The total cost H will be the cost of all the controllers:

$$H = \sum_{i=1}^{k}\sum_{j \in C_i} d(w_j, c_i) \times wt_j = \sum_{i=1}^{k} H_i \tag{9}$$

Similar to load balance ΔL, the lower the cost H, the better the performance, and the objective can be defined as:

$$\min\ \alpha\Delta L + \beta H \tag{10}$$

$$\text{subject to:} \quad \alpha + \beta = 1, \quad L_i \leq B_{\max} \tag{11}$$

To solve the CPP, we employ the Deep reinforcement learning technique actor-critic method. Actor and critic are the two primary components of the Actor-Critic algorithm. The actor is responsible for choosing actions based on the current condition of the environment, while the critic evaluates the chosen actions and gives the actor feedback. This feedback is utilized to revise the actor's policy in order to enhance decision-making over time.

In the context of controller placement in SDN, the actor represents the policy that selects the optimal placement of controllers, while the critic estimates the value or quality of the chosen placements. The critic provides feedback on the effectiveness of controller placements, such as load balancing and network performance metrics, to guide the actor in making better decisions. The proposed learning includes the following set of state, action, and reward.

- State:
  The state represents the current information or observation of the SDN environment. It includes relevant network characteristics, such as network topology, traffic patterns, link utilization, controller and switch loads, and other contextual information that can influence the decision-making process for controller placement. The state provides the necessary context for the actor to make informed decisions.

$$s_{c_j} = (t_{P_i}(j), F_{P_i}(j)), j \in \mathbb{N}, j \leq k$$
$$S = (s_{c_1}, s_{c_2}, \ldots, s_{c_k})$$

- Action
  The action corresponds to the selection of controller placements in the SDN network. Each action represents a potential configuration of controller nodes' locations within the network topology. The actor, guided by its policy, chooses the action that it believes will lead to load balancing and improved network performance.

$$a_{w_i} = (w_i^1, w_i^2, \ldots, w_i^k), j \in \mathbb{N}, j \leq k$$
$$A = (a_{w1}, a_{w2}, \ldots, a_{wn})$$

$$w_{ij} = \begin{cases} 1, & \text{if} w_i \in C_j \\ 0, & \text{otherwise} \end{cases}.$$

- Reward
  The reward serves as feedback to the actor regarding the quality or desirability of the chosen controller placements. It is a scalar value that quantifies how well the chosen action aligns with the objective of load balancing in the SDN network.As we have to maximize the reward and minimize the load and cost associated, it will be

$$r = -(\alpha\Delta L + \beta/\delta)$$

  where δ is normalizing factor to balance both the values, as they might be out of order.

The actor's policy is updated based on the critic's evaluation of the chosen action and the received reward. The critic estimates the value or expected return associated with the chosen action and provides this information to the actor. The actor then adjusts its policy using techniques like policy gradients or advantage estimation, aiming to increase the likelihood of selecting actions that lead to higher rewards in future states. By incorporating the state, action, and reward within the Actor-Critic framework, the paper seeks to train an actor network that can learn optimal controller placement policies for load balancing in SDN. The actor utilizes the critic's value estimation and received rewards to iteratively improve its policy over time, leading to more effective controller placement strategies that enhance network performance and achieve load balancing objectives in SDN environments.

This pseudocode outlines the basic framework of the Actor-Critic algorithm for training the actor and critic networks in the context of load balanced controller placement in SDN. It captures the core steps involved in interacting with the SDN environment, updating the networks based on the observed experiences, and gradually improving the controller placement policy to maximize cumulative rewards.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:06:27 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1 DRL for CPP

Initialize θ for acotr and ϕ for critic network.
Initialize the replay buffer D to store the experiences.
Set the learning rate α_θ and α_ϕ for the actor and critic networks, respectively.
for each episode **do**
  Reset the environment and get the observed state.
  for each time step **do**
    Select an action a_t from the actor network with the current state s_t
    Execute the action a_t in the SDN environment
    Observe the next state s_{t+1} and the immediate reward r_t
    Store the experience (s_t, a_t, r_t, s_{t+1}) in the replay buffer D.
    Sample a mini-batch of experiences from the replay buffer D.
    Update the critic network by minimizing the mean-squared error
    Update the actor network by performing gradient ascent on the expected return
    $\nabla_{\theta_{\text{actor}}} J \approx \frac{1}{N} \sum_i \nabla_{\theta_{\text{actor}}} Q(s_i, a_i|\theta_Q)$
    Update the target critic network parameters
    Update the state s_t next to s_{t+1}
    OUTPUT: Return the learned actor network π and the critic network V.
  end for
end for

Algorithm 2 DRL for CPP

Input: State s
Output: Action a
Initialize actor-network with random weights
Initialize replay buffer D
Initialize target actor-network with the same weights as actor-network
for episode = 1 to N **do**
  Initialize environment and observe initial state s_0
  for t = 1 to T **do**
    Select action a_t using the actor-network with exploration noise
    Execute action a_t and observe reward r_t and next state s_{t+1}
    Store transition (s_t, a_t, r_t, s_{t+1}) in replay buffer D
    if replay buffer D contains sufficient samples **then**
      Sample a random minibatch of transitions (s_i, a_i, r_i, s_{i+1}) from D
      Compute target action a'_i using the target actor-network and next state s_{i+1}
      Compute target Q-value $y_i = r_i + \gamma Q(s_{i+1}, a'_i|\theta_Q)$
      Update the actor-network using the gradient:
      $\nabla_{\theta_{\text{actor}}} J \approx \frac{1}{N} \sum_i \nabla_{\theta_{\text{actor}}} Q(s_i, a_i|\theta_Q)$
      Update the target actor network weights:
      $\theta_{\text{target_actor}} \leftarrow \tau\theta_{\text{actor}} + (1-\tau)\theta_{\text{target_actor}}$
    end if
  end for
end for

IV. Experiment Details

Software Defined Network is simulated using $mininet$ simulator with two different types of zoo topologies in India and the USA, which have different numbers of routers in main cities with latitude and longitude information. $Haversine$ algorithm is used to find the distance between nodes using latitude and longitude. These topologies help to create real-time network traffic and its dynamic load patterns given in Fig. 2.

TABLE I
Simulation Parameters

Parameter/Setting	Values
Simulated Environment	Mininet
Packet-in flows	10% (total flows to switch)
Packet loss	5%
Number of controllers	2,4
Number of nodes	India - 145; USA - 68
Number of topologies	2

A. Evaluation

The general metrics used to evaluate DRL are Reward, Average, and Discounted Reward. The simulated results are plotted over episodes as the X-axis and reward as the Y-axis for different controllers and topologies. In Fig. 3, load balance over episodes in the Indian topology for controllers is simulated using DRL; as a result, we can infer that the rewarded load balance factor is optimized over the episodes. Similarly, in the plot for topologies, the reward is maximized, i.e. load balance is minimized over episodes

In Fig. 4 for over a range of α and β if we see the Load Deviation, for the small α values, the change in deviation of the load is minimum which implies the distribution of load is evenly balanced and it disrupts over the increase in α Similarly from the plots of rewarded cost over episodes will imply that reward is maximized implying cost is minimized over the time. Fig. 5 demonstrates the load balance performance that is represented as negative cost -H over 100 episodes for systems with 2 and 4 controllers. It is observed that using 4 controllers achieves slightly better and more stable load balancing compared to 2 controllers, as reflected in the smoother and higher running average of rewards over episodes. Fig. 6 compares the negative cost H across 200 episodes for two different network topologies India and USA. The network topology influences the learning dynamics and cost optimization that is demonstrated in Indian topology shows a steady improvement and stabilization of the reward, indicating effective cost reduction. In contrast, the USA topology starts off better but then slightly degrades after 150 episodes.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:06:27 UTC from IEEE Xplore. Restrictions apply.

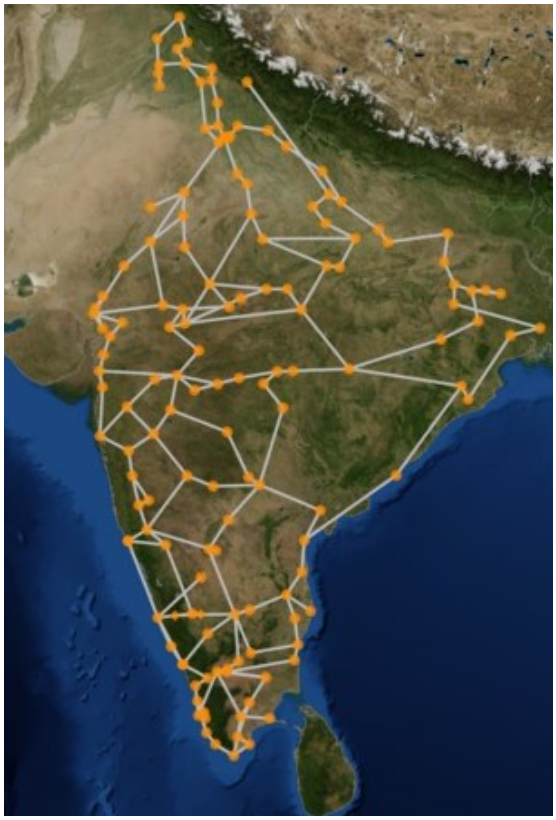
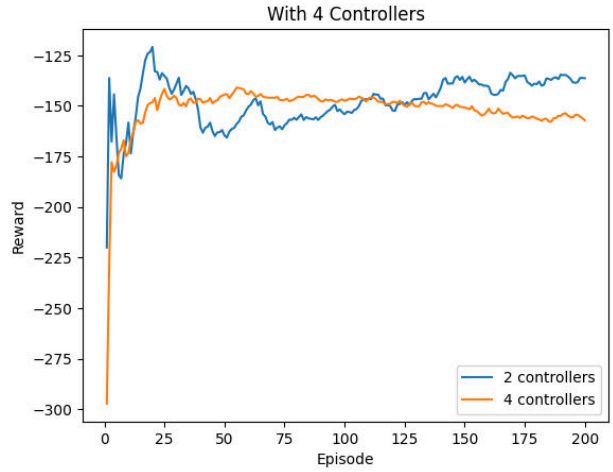
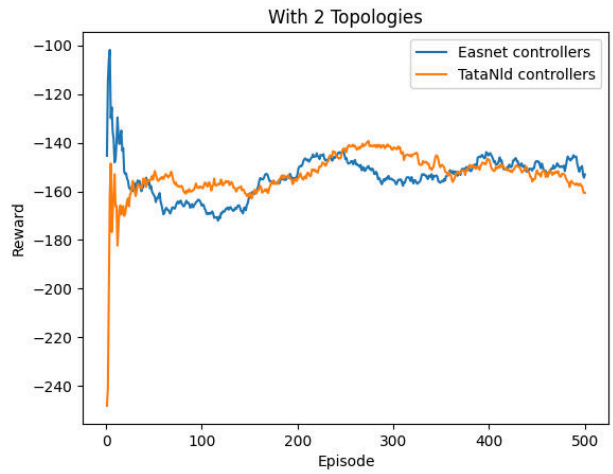


Fig. 2. The Indian topological zoo used in the simulation contains 145 nodes



(a) Controllers



(b) Topologies

Fig. 3. Cummulative reward $-(\alpha\Delta L + \beta H/\delta)$ over varying episodes for different controllers and different topologies

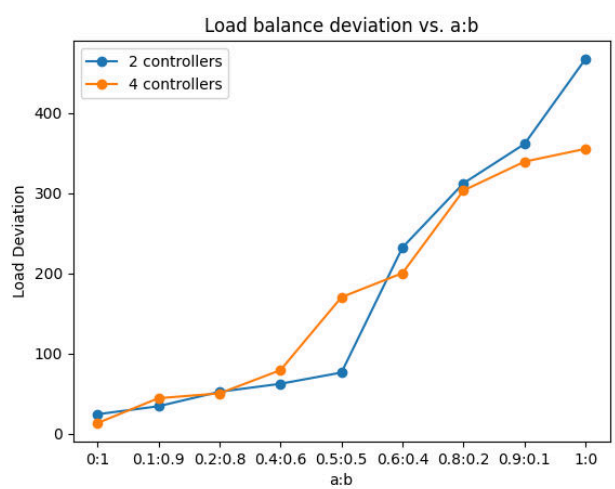


Fig. 4. Load Balance (ΔL) over $\alpha : \beta$ for DRL and K-means

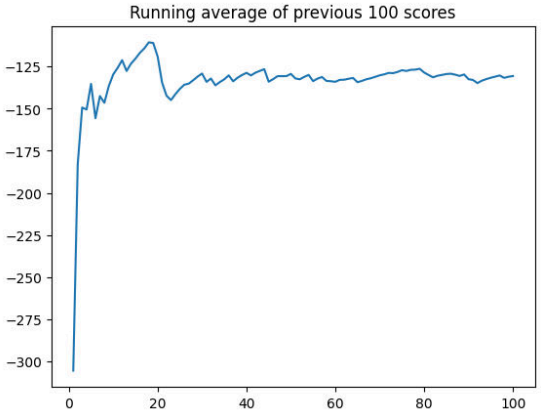
V. Conclusion and Future Scope

In conclusion, this paper focused on the application of Deep Reinforcement Learning (DRL) for the Load Balanced Controller Placement Problem in Software-Defined Networking (SDN). The Actor-Critic algorithm was employed to learn optimal controller placement policies that aim to achieve load balancing and enhance network performance. Through extensive experimentation using Mininet and two Zoo topologies, the effectiveness of the proposed approach was evaluated. The results demonstrated that the Actor-Critic algorithm, coupled with DRL techniques, offers promising solutions for load balanced controller placement in SDN. The algorithm successfully learned optimal policies that improved load balancing, and cost optimization. By adaptively adjusting controller placements based on real-time feedback, the algorithm dynamically responded to changes in network conditions, workload demands, and topology variations. The experimental analysis highlighted the benefits of distributed controller placement strategies in achieving load balancing and enhancing network performance compared to centralized approaches.

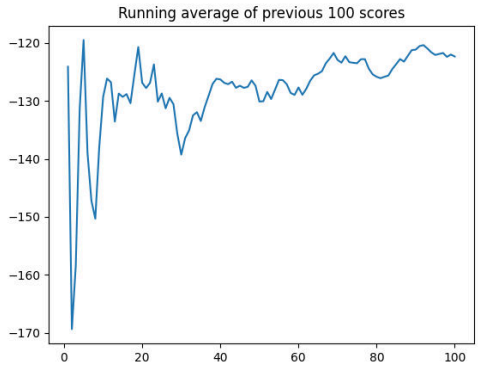
VI. Acknowledgments

This work is supported by the Department of Science and Technology (DST), Government of India, under grant number DST/TSG/NTS/2013/26 and EEQ/2018/001482.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:06:27 UTC from IEEE Xplore. Restrictions apply.

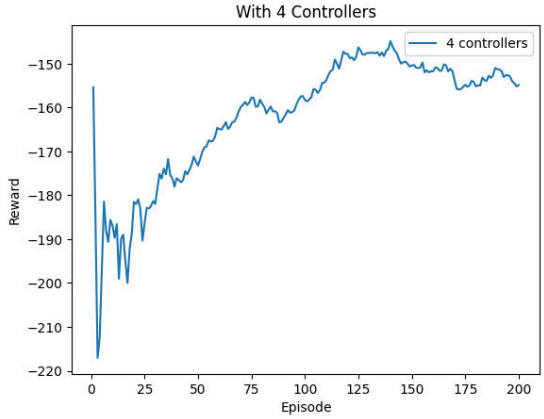


(a) 2 Controllers

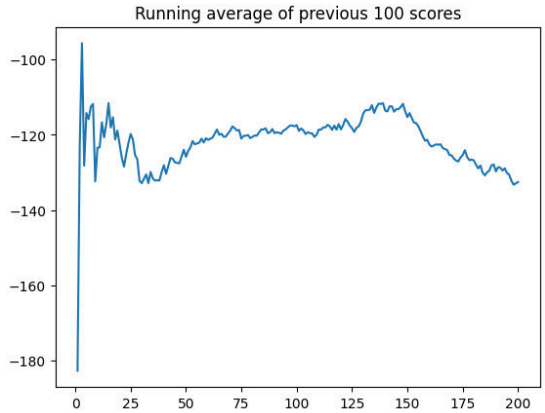


(b) 4 Controllers

Fig. 5. Load balance $(-H)$ over 100 episodes for different controllers and different topologies



(a) Indian Topology



(b) USA Topology

Fig. 6. Cost $(-H)$ over 200 episodes for different topologies

REFERENCES

[1] Y. Wu, S. Zhou, Y. Wei, and S. Leng, "Deep reinforcement learning for controller placement in software defined network," in *IEEE INFOCOM 2020 - IEEE Conference on Computer Communications Workshops (INFOCOM WKSHPS)*, pp. 1254–1259, 2020.

[2] K. Yuan Tan, S. Chin Tan, and T. Chee Chuah, "A multi-phase drl-driven sdn migration framework addressing budget, legacy service compatibility, and dynamic traffic," *IEEE Access*, vol. 13, pp. 33202–33219, 2025.

[3] S. Mohanty, A. S. Shekhawat, B. Sahoo, H. K. Apat, and P. Khare, "Minimizing latency for controller placement problem in sdn," in *2021 19th OITS International Conference on Information Technology (OCIT)*, pp. 393–398, 2021.

[4] B. Isong, R. R. S. Molose, A. M. Abu-Mahfouz, and N. Dladlu, "Comprehensive review of sdn controller placement strategies," *IEEE Access*, vol. 8, pp. 170070–170092, 2020.

[5] J. Lu, Z. Zhang, T. Hu, P. Yi, and J. Lan, "A survey of controller placement problem in software-defined networking," *IEEE Access*, vol. 7, pp. 24290–24307, 2019.

[6] D. Kreutz, F. M. V. Ramos, P. E. Veríssimo, C. E. Rothenberg, S. Azodolmolky, and S. Uhlig, "Software-defined networking: A comprehensive survey," *Proceedings of the IEEE*, vol. 103, no. 1, pp. 14–76, 2015.

[7] A. Nahar, N. Bhardwaj, D. Das, and S. K. Das, "A hypergraph approach to deep learning based routing in software-defined vehicular networks," *IEEE Transactions on Mobile Computing*, vol. 24, no. 5, pp. 3844–3859, 2025.

[8] Y. Fu, Q. Kang, J. Wang, and Y. Gu, "Sdn controller deployment method with capacity and control link reliability constraints," in *2021 International Conference on Wireless Communications and Smart Grid (ICWCSG)*, pp. 153–158, 2021.

[9] W.-K. Lai, Y.-C. Wang, Y.-C. Chen, and Z.-T. Tsai, "Tssm: Time-sharing switch migration to balance loads of distributed sdn controllers," *IEEE Transactions on Network and Service Management*, vol. 19, no. 2, pp. 1585–1597, 2022.

[10] K. Saeed and M. O. Ullah, "Toward reliable controller placements in software-defined network using constrained multi-objective optimization technique," *IEEE Access*, vol. 10, pp. 129865–129883, 2022.

[11] M. R. Abedi, N. Mokari, M. R. Javan, H. Saeedi, E. A. Jorswieck, and N. Zorba, "Low complexity and mobility-aware robust radio, storage, computing, and cost management for cellular vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 74, no. 2, pp. 3327–3344, 2025.

[12] S. Wu, X. Chen, L. Yang, C. Fan, and Y. Zhao, "Dynamic and static controller placement in software-defined satellite networking," *Acta Astronautica*, vol. 152, pp. 49–58, 2018.

[13] C. Xu, C. Xu, B. Li, S. Li, and T. Li, "Load-aware dynamic controller placement based on deep reinforcement learning in sdn-enabled mobile cloud-edge computing networks," *Computer Networks*, vol. 234, p. 109900, 2023.

[14] J. Guo, L. Yang, D. Rincx00F3;n, S. Sallent, Q. Chen, and X. Liu, "Static placement and dynamic assignment of sdn controllers in leo satellite networks," *IEEE Transactions on Network and Service Management*, pp. 1–1, 2022.

[15] L. Yao, P. Hong, and W. Zhou, "Evaluating the controller capacity in software defined networking," in *2014 23rd International Conference on Computer Communication and Networks (ICCCN)*, pp. 1–6, 2014.

Authorized licensed use limited to: University of Zululand. Downloaded on March 25,2026 at 01:06:27 UTC from IEEE Xplore. Restrictions apply.